



TRAINING PROGRAMME



Lesson 2 & 3

Code Smells

*"A person who never made a mistake,
never tried anything new."— Albert Einstein*

Code smells

- A *code smell* is a surface indication, a symptom - that something is not quite right in the system.
- It is not inherently bad on its own, but it's a clue suggesting to investigate potential design problems.
- Study the recurring types of smells is an effective way to identify the issues and solve them as early as possible, hence in the refactoring phase.



Code smells

Potential problems:

→ Rigidity

- ◆ When the software is difficult to change. A small change triggers a cascade of changes.

→ Fragility

- ◆ When a single change breaks the software in many different places.

→ Immobility

- ◆ When code is not reusable.

→ Viscosity of Design

- ◆ When design is not simple enough it introduces the hazard of taking shortcuts and introducing technical debt.

→ Viscosity of Environment

- ◆ When the environment is not effective enough (slow, flaky tests, clunky CI, test environments limitations, etc.) it introduces the hazard of not doing those activities properly by everyone and introducing technical debt as a consequence



Code smells

What can we do to go in the right direction? Follow the **core principles...**

- **KISS** (keep it simple, simian!)
- **DRY** (don't repeat yourself)
- **YAGNI** (you aren't gonna need it)
- **SOLID**
- **4 RULES OF SIMPLE DESIGN**
- **BALANCED ABSTRACTION PRINCIPLE**
- **Tell don't ask** (Law of Demeter)
- **LEAST ASTONISHMENT (WTF PRINCIPLE)**



Code smells

What do we want to achieve following those principles?

→ Maximize **Cohesion**

- ◆ Cohesion is a metric telling **how strongly related and coherent** are the **responsibilities within the classes** of an application

→ Minimize **Coupling**

- ◆ Coupling is a metric for measuring **the degree of interdependence between the classes** of an application

→ Optimize **Connascence**

- ◆ Connascence is an alternative, extended taxonomy for OO, extending both Coupling & Cohesion ideas

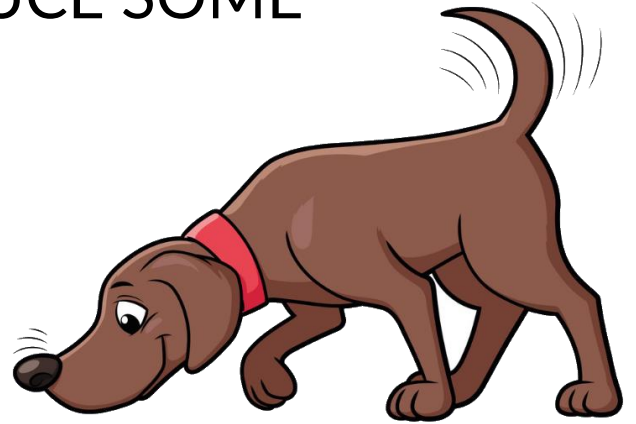


Code smells

What happens if we ignore those principles?

WE ARE GOING TO INTRODUCE SOME

**CODE
SMELL**



Code smells: BLOATERS

→ Long Parameter List

0 (*niladic*) => **Ideal**, 1 (*monadic*) => **Ok**, 2 (*dyadic*) => **Acceptable**, 3 (*triadic*) => Debatable (but we **avoid it**), 3+ (*polyadic*) => **Only with special justification**. **Coupling violation**. **Connascence of position**.

→ Data Clumps

Same data items together in lots of places. Special case of duplicated code. **DRY violation**. **Cohesion violation**.

→ Primitive Obsession

Don't use primitive types as substitutes for classes. If the data type is sufficiently complex, use a class to represent it. **Coupling violation**. **Connascence of meaning**.



Code smells: BLOATERS

→ Long Method

Methods should do only one thing. Should do it well. Should do it only. One level of abstraction. No more than 10-15 lines.

SRP violation. Cohesion violation.

→ Large Class

Classes should have only one responsibility. No more than 50 lines per class. **SRP violation. Cohesion violation.**



Code smells: CHANGE PREVENTERS

→ Divergent Change (aka Excessive Cohesion)

When one class is commonly changed in different ways for different reasons (God class). **OCP, SRP violation. Cohesion violation.**

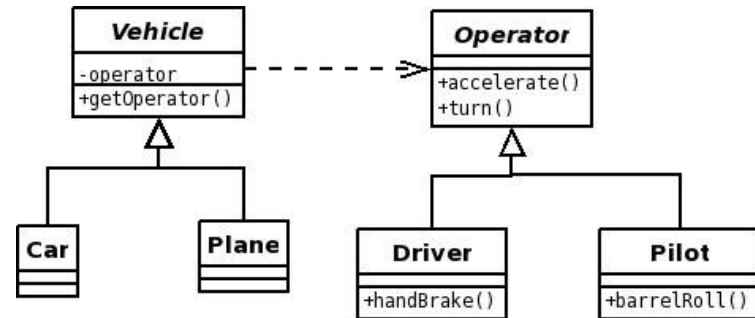
→ Shotgun Surgery (aka Excessive Decoupling)

Opposite of Divergent change. One change, forces lots of little changes in different classes. *Spread Concerns, Over Layering and Copy-pasting.*

DRY violation. Coupling violation.

→ Parallel Inheritance Hierarchies

Special case of shotgun surgery. Creating a subclass of one class, forces subclass of another.



Code smells: OO Abusers

→ Refused Bequest

When a subclass uses only some of the methods and properties inherited from its parents. Usually done for functionality reuse instead then modelling correct abstraction. **LSP violation.**

→ Alternative Classes with Different Interfaces

If two classes are similar on the inside, but different on the outside, perhaps they can be modified to share a common interface.



Code smells: OO Abusers

→ Switch Statements

Can lead to same switch statement scattered in different places.

Can lead to **Dry violation** or **Cohesion violation**.

→ Temporary Field

Class contains an instance variable set only in certain circumstances. **Cohesion violation**.



Code smells: COUPLERS

→ Middle Man

If a class is delegating all its work, cut out the middleman. Beware classes that are wrappers over other classes or existing functionality. Special case of lazy class. **Cohesion violation.**

→ Message Chains

Too many dots: Dog.Body.Tail.Wag()

Should be: Dog.ExpressHappiness()

Law of **Demeter** violation. **Coupling** violation.



Code smells: COUPLERS

→ Feature Envy

A class that uses public methods or fields of another class excessively.

Tell, don't ask violation. Cohesion and Coupling violation.

→ Inappropriate Intimacy

A class that uses protected or internal methods or fields of another class excessively. Special case of **feature envy** especially with inheritance.

Cohesion violation.



Code smells: DISPENSABLES

→ Lazy Class

A class that does too little. May be acting only as middle man or a data class or can be caused by speculative generality.

→ Data Class

Classes that have fields, properties, and nothing else. Anaemic classes that contain no behaviour. Special case of lazy class.

Cohesion violation.

→ Speculative Generality

There is an unused class, method, field or parameter. **YAGNI violation.**

Cohesion violation.



Code smells: DISPENSABLES

→ Comments

Make effort to create code that expresses intent instead of comments.

KISS violation.

→ Dead Code

Code that has no references to it, commented code. Remember that
“Deleted code has no bugs and improves readability”

→ Duplicated Code

When identical or very similar code exists in more than one location or duplicated knowledge. **DRY violation. Cohesion violation.**



Object Calisthenics and Code smells

Object calisthenics	→	Code smells
Only one level of indentation per method		Long Method
Don't use the ELSE keyword		Long Method / Duplicated Code
Wrap all primitives and strings		Primitive Obsession
First class collections		Divergent Change / Large Class
One dot per line		Message Chains
Keep all entities small		Large Class / Long Method / Long Parameter List
No classes with more than two instance variables		Large Class
No getters / setters / properties		Feature Envy
All classes must have state, no static methods, no utility classes		Lazy Class / Middle man / Feature envy



Code smells



KATA – Smelling Tic Tac Toe

Find and clean up all the smells from this repository:

<https://github.com/AgileTechPraxis/CodeSmells>

